

This command has a variety of sub commands, many of which should look fairly familiar at this point. The `list` sub command acts in a manner very similar to `list` sub commands we have discussed previously, but this one only lists the installed system packages. This sub command is often used with `grep` to help you find out if a particular package is installed.

Once you have located an installed package you can then have it list the executables contained by the package using the `executables` command. This is much more practical than finding the executables manually. The sub command takes in a single argument which is the package name. The `executables` command has a single optional argument, `--full-path`, which will output the full path to all the executable programs. The example below shows how to use these commands to check the path for all of the `turtlesim` executables:

```
$ ros2 pkg list | grep turtle
turtlesim
$ ros2 pkg executables turtlesim --full-path
/opt/ros/eloquent/lib/turtlesim/draw_square
/opt/ros/eloquent/lib/turtlesim/mimic
/opt/ros/eloquent/lib/turtlesim/turtle_teleop_key
/opt/ros/eloquent/lib/turtlesim/turtlesim_node
```

If you just wanted to know the path to the `turtlesim` executables you could use the `prefix` sub command, which returns the path for a given package's executables.

Each ROS package contains an XML file that contains metadata for the package, including information such as the license, maintainer, and its dependencies. ROS `pkg` has a handy `xml` sub command to print these files to the screen, saving you the hassle of locating and opening the file. You can use `grep` on the output of this command to get just the info you need. Below is an example of `xml` and `prefix` used to find the directory of `turtlesim`, its maintainer, and its license:

```
$ ros2 pkg prefix turtlesim
/opt/ros/eloquent
$ ros2 pkg xml turtlesim | grep maintainer
<maintainer email="dthomas@osrfoundation.org">Dirk Thomas</maintainer>
$ ros2 pkg xml turtlesim | grep license
<license>BSD</license>
kscottz@kscottz-ratnest:~$
```

The last sub command in the `pkg` command is `create`. `create` is a tool to help you create a ROS package. We'll use this sub command later in the chapter to create a new ROS package. The short of it is that you feed the command your package name and all of the relevant information for your package as optional arguments.

ROS 2 Security

Found a bug? [Edit this page on GitHub](#).

The ROS API

ROS comprises many software libraries that provide a wide array of functionality useful when building robot applications. The libraries you need will depend on the details of your project. In this section we will introduce two core libraries that you are likely to interact with frequently when developing with ROS:

- `rclpy` : Python client library
- `rclcpp` : C++ client library

A ROS *client library* provides the data structures, functions, and syntactic sugar that make it convenient to develop in a particular programming language. Here we will cover just the Python and C++ libraries because they're the most widely used. But you can find ROS client libraries for many other languages, from Ada to JavaScript to Rust, and beyond.

Note: In this section we aim for a gentle and efficient introduction to the ROS API. In service of that goal, we will purposefully ignore and/or violate various conventions and patterns.

Publishing and Subscribing to Topics in Python

Publishing data with ROS is easy. Here is a complete Python program that publishes string messages:

```
from time import sleep
import rclpy
from std_msgs.msg import String

rclpy.init()
node = rclpy.create_node('my_publisher')
pub = node.create_publisher(String, 'chatter', 10)
msg = String()
i = 0
while rclpy.ok():
    msg.data = f'Hello World: {i}'
    i += 1
    print(f'Publishing: "{msg.data}"')
    pub.publish(msg)
    sleep(0.5)
```

Try it out yourself. (Make sure that in every shell used you have sourced your ROS setup file as we discussed in the previous chapter; e.g., `source /opt/ros/foxy/setup.bash`.) Copy the code block above into a file, call it `talker.py`, then feed it to your Python3 interpreter:

```
$ python3 talker.py
```

You should see:

```
Publishing: "Hello world: 0"  
Publishing: "Hello world: 1"  
Publishing: "Hello world: 2"
```

It prints to console, but is the data going anywhere? We can check our work using the `ros2 topic` tool that was introduced earlier. In another shell (leave your talker running), run:

```
$ ros2 topic echo chatter
```

You should see the following, though the numbers will vary depending on timing between the two commands:

```
data: 'Hello world: 13'  
---  
data: 'Hello world: 14'  
---  
data: 'Hello world: 15'
```

So we have a working talker. Now we can add our own listener to use in place of `ros2 topic`. Here is a complete Python program that subscribes to string messages and prints them to console:

```
import rclpy  
from std_msgs.msg import String  
  
def cb(msg):  
    print(f'I heard: "{msg.data}"')  
  
rclpy.init()  
node = rclpy.create_node('my_subscriber')  
sub = node.create_subscription(String, 'chatter', cb, 10)  
rclpy.spin(node)
```

Try it out yourself. Copy the code block above into a file and call it `listener.py`. With your talker still running in one shell, start up your listener in another shell:

```
$ python3 listener.py
```

You should see (again, numbers will vary depending on timing):

```
I heard: "Hello world: 35"  
I heard: "Hello world: 36"  
I heard: "Hello world: 37"
```

Digging into the Python Code

Now that we know these programs work, we can dig into their code. Both programs start with the same preamble:

```
import rclpy  
from std_msgs.msg import String
```

We need to import the `rclpy` client library, which gives us much of what we need to write ROS applications in Python. But we also need to specifically import the ROS message type(s) that we will use. In this case we are using the simple `std_msgs/String` message, which contains a single field called `data`, of type `string`. If we wanted to use the `sensor_msgs/Image` message, which represents camera images, then we would write `from sensor_msgs.msg import Image`.

After the imports, both programs perform common initialization:

```
rclpy.init()  
node = rclpy.create_node('my_node_name')
```

We initialize the `rclpy` library and then call into it to create a `Node` object, giving it a name. Subsequently we will operate on that `Node` object.

In the talker, we use the `Node` object to create a `Publisher` object:

```
pub = node.create_publisher(String, 'chatter', 10)
```

We declare the type of data we will publish (`std_msgs/String`), the name of the topic on which we will publish (`chatter`), and the maximum number of outbound messages to locally queue up (10). That last argument comes into play when we are publishing faster than subscribers are consuming the data.

The equivalent step in the listener is to create a `Subscription` object:

```
sub = node.create_subscription(String, 'chatter', cb, 10)
```

The type (`String`) and topic name (`chatter`) arguments have the same meaning as the `create_publisher()` call, and the final argument (10) is setting an analogous maximum queue size for inbound messages. The key difference is the `cb` argument, which refers to this *callback* function that we also defined in the listener:

```
def cb(msg):
    print(f'I heard: "{msg.data}"')
```

That function will be called whenever the listener receives a message, and the received message will be passed in as an argument. In this case we simply print the content to console.

With the callback defined and the `Subscription` created, the rest of the listener is one line:

```
rclpy.spin(node)
```

This call hands control over to `rclpy` to wait for new messages to arrive (and more generally for events to occur) and invoke our callback.

Back in the talker, we create a simple loop to use our `Publisher`:

```
msg = String()
i = 0
while rclpy.ok():
    msg.data = f'Hello World: {i}'
    i += 1
    print(f'Publishing: "{msg.data}"')
    pub.publish(msg)
    sleep(0.5)
```

These steps are clear enough: we create a message object and then on each iteration of the loop, we update the message content and publish it, sleeping briefly between iterations.

Publishing and Subscribing to Topics in C++

Now we will write the same talker and listener pair, this time in C++.

Here is a complete C++ program that publishes string messages:

```

#include <unistd.h>
#include <iostream>
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    auto node = rclcpp::Node::make_shared("minimal_publisher");
    auto pub = node->create_publisher<std_msgs::msg::String>("chatter", 10);
    std_msgs::msg::String message;
    auto i = 0;
    while (rclcpp::ok()) {
        message.data = "Hello world: " + std::to_string(i++);
        std::cout << "Publishing: " << message.data << std::endl;
        pub->publish(message);
        usleep(500000);
    }
    return 0;
}

```

Of course, as for all C++, we need to compile this program. Managing the compilation arguments for C++ is cumbersome, so we use CMake to help. Here is the complete CMake code that allows us to build the talker example:

```

cmake_minimum_required(VERSION 3.5)
project(talker_listener)

find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)

add_executable(talker talker.cpp)
target_include_directories(talker PRIVATE ${rclcpp_INCLUDE_DIRS}
${std_msgs_INCLUDE_DIRS})
target_link_libraries(talker ${rclcpp_LIBRARIES} ${std_msgs_LIBRARIES})

```

Try it out yourself. Copy the C++ code into a file called `talker.cpp` and the CMake code into a file called `CMakeLists.txt`. Have them side-by-side in a directory and then invoke `cmake` followed by `make`:

```

$ cmake .
$ make

```

You should end up with a compiled executable called `talker`. Run it:

```

$ ./talker

```

You should see:

```
Publishing: "Hello world: 0"
Publishing: "Hello world: 1"
Publishing: "Hello world: 2"
```

Keep the talker running and another shell try `ros2 topic` to listen in:

```
$ ros2 topic echo chatter
```

You should see (numbers will vary depending on timing between the two commands):

```
data: 'Hello world: 13'
---
data: 'Hello world: 14'
---
data: 'Hello world: 15'
```

Now we can write our own listener to use in place of `ros2 topic`. Here is a complete C++ program that subscribes to string messages and prints them to console:

```
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

void cb(const std_msgs::msg::String::SharedPtr msg)
{
    std::cout << "I heard: " << msg->data << std::endl;
}

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    auto node = rclcpp::Node::make_shared("my_subscriber");
    auto sub = node->create_subscription<std_msgs::msg::String>("chatter", 10, cb);
    rclcpp::spin(node);
    return 0;
}
```

Copy the code block into a file called `listener.cpp`. To arrange for it to be compiled, we also need to add some corresponding CMake code to the bottom of our `CMakeLists.txt` file from earlier:

```
add_executable(listener listener.cpp)
target_include_directories(listener PRIVATE ${rclcpp_INCLUDE_DIRS}
${std_msgs_INCLUDE_DIRS})
target_link_libraries(listener ${rclcpp_LIBRARIES} ${std_msgs_LIBRARIES})
```

Configure and build again:


```
$ cmake .  
$ make
```

Now you should have also have a `listener` executable. With your talker still running in one shell, start up your listener in another shell:

```
$ ./listener
```

You should see (again, numbers will vary depending on timing):

```
I heard: "Hello world: 35"  
I heard: "Hello world: 36"  
I heard: "Hello world: 37"
```

Digging into the C++ Code

Now that we know these programs work, we can dig into their code. Both programs start with the same preamble:

```
#include "rclcpp/rclcpp.hpp"  
#include "std_msgs/msg/string.hpp"
```

We always need to include the `rclcpp` client library, which gives us much of what we need to write ROS applications in C++. But we also need to specifically import the ROS message type(s) that we will use. In this case we are using the simple `std_msgs/String` message, which contains a single field called `data`, of type `string`. If we wanted to use the `sensor_msgs/Image` message, which represents camera images, then we would `#include "sensor_msgs/msg/image.hpp"`.

After the imports, both programs perform common initialization:

```
rclcpp::init(argc, argv);  
auto node = rclcpp::Node::make_shared("my_node_name");
```

We initialize the `rclcpp` library and then call into it to create a `Node` object, giving it a name. Subsequently we will operate on that `Node` object.

In the talker, we use the `Node` object to create a `Publisher` object:

```
auto pub = node->create_publisher<std_msgs::msg::String>("chatter", 10);
```

We declare via template the type of data we will publish (`std_msgs/String`), the name of the topic on which we will publish (`chatter`), and the maximum number of outbound messages to locally queue up (10). That last argument comes into play when we are publishing faster than subscribers are consuming the data.

The equivalent step in the listener is to create a `Subscription` object:

```
auto sub = node->create_subscription<std_msgs::msg::String>("chatter", 10, cb);
```

The type (`String`) and topic name (`chatter`) arguments have the same meaning as for the `create_publisher()` call, and the numerical argument (10) is setting an analogous maximum queue size for inbound messages. The key difference is the `cb` argument, which refers to this *callback* function that we also defined in the listener:

```
void cb(const std_msgs::msg::String::SharedPtr msg)
{
    std::cout << "I heard: " << msg->data << std::endl;
}
```

That function will be called whenever the listener receives a message, and the received message will be passed in as an argument. In this case we simply print the content to console.

With the callback defined and the `Subscription` created, the rest of the listener is one line:

```
rclcpp::spin(node);
```

This call hands control over to `rclcpp` to wait for new messages to arrive (and more generally for events to occur) and invoke our callback.

Back in the talker, we create a simple loop to use our `Publisher` :

```
std_msgs::msg::String message;
auto i = 0;
while (rclcpp::ok()) {
    message.data = "Hello world: " + std::to_string(i++);
    std::cout << "Publishing: " << message.data << std::endl;
    pub->publish(message);
    usleep(500000);
}
```

In these steps we create a message object, and then on each iteration of the loop we update the message content and publish it, sleeping briefly between iterations.